Calibration

SWAT+ Documentation

docs.swat.tamu.edu

Contents

Pages . . . 2
Components . . . 2
HRU water balance . . . 2
Channel sediment . . . 3
Plant growth . . . 3
Outputs . . . 3
Input files . . . 3
Related pages . . . 3
Source . . . 4
Workflow . . . 4
Object types . . . 4
Re-initialization . . . 5
Related pages . . . 5
Purpose . . . 5
Source . . . 5
cal_parms.cal . . . 5
calibration.cal . . . 6
Carbon parameters . . . 7
Related files . . . 15
Source . . . 15
Two condition syntaxes . . . 15
Supported variables . . . 15
Regionalization . . . 16
Decision-table conditional updates . . . 16
Parameter dispatch . . . 16
Related pages . . . 16
Pattern . . . 16
sensitivityAPI . . . 16
Roll your own . . . 17
Related pages . . . 17

SWAT+ supports two calibration modes that are configured and run from inside the model.

- **Hard calibration** fits named model parameters to observed time series (streamflow, sediment, nutrient loads). Parameter values are perturbed within absolute bounds, the model is re-run, and the simulation is compared with measured data. Implemented in `calhard_*.f90`. Driven by `cal_parms.cal` (parameter definitions and bounds) and `calibration.cal` (the change list applied each run).
- **Soft calibration** adjusts process outputs toward regional water-balance and yield targets without exposing the underlying parameters as a sample space. Targets are expressed as fractions of precipitation for the water balance, channel sediment budgets, and plant growth. Implemented in `calsoft_*.f90`. Driven by `codes.sft`, `wb_parms.sft`, `water_balance.sft`, `ch_sed_budget.sft`, `ch_sed_parms.sft`, `plant_parms.sft`, and `plant_gro.sft`.

The two modes share the same internal parameter selector (`cal_parm_select.f90`) and the same conditional change machinery (`cal_conditions.f90`).

Pages

- Hard calibration. Parameter perturbation, sources, the role of `calibration.cal`.
- Soft calibration. Regional water balance and yield targets, sources, the role of `codes.sft`.
- Parameter change files. Format of `cal_parms.cal` and `calibration.cal`, verified against the readers.
- Conditions and regionalization. Applying a change only to HRUs that match a condition (HSG, texture, plant, land use, calibration group).
- Sensitivity workflow. Driving SWAT+ from an external tool to compute sensitivity indices, using the same `cal_parms.cal` to define the sample space.

Soft calibration nudges model outputs toward regional targets expressed as fractions of precipitation (water balance), per-stream-order channel sediment budgets, and per-region plant yields. The targets are user-supplied and the model adjusts a small fixed set of parameters internally until each component falls inside its target window. Soft calibration is run separately from a normal simulation; it loops over the relevant land-use regions and process components, updates parameters, and re-runs the affected balance routine.

Components

The soft codes file (`codes.sft`) is read by `calsoft_read_codes.f90` into `cal_codes` (`type soft_calibration_codes` in `calibration_data_module.f90`). Each switch enables one component of the soft calibration:

Code	Switch	Component	Driver routine
`hyd_hru`	`n`, `a`, `b`, `y`	HRU water balance by land use in each region. `a` calibrates all components (ET, surface runoff, lateral flow, percolation). `b` calibrates baseflow and total runoff only.	`calsoft_hyd.f90`, `calsoft_hyd_bfr.f90`
`hyd_hrul`	`y/n`	`hru_lte` water balance by land use in each region	`caltsoft_hyd.f90`
`plt`	`y/n`	Plant growth (LAI, harvest index) by region	`calsoft_plant.f90`
`sed`	`y/n`	Upland sediment yield by land use in each region	`calsoft_sed.f90`
`nut`	`y/n`	HRU nutrient balance by land use in each region	(gated in `calsoft_control.f90`)
`chsed`	`y/n`	Channel widening and bank accretion by stream order	(gated in `calsoft_control.f90`)
`chnut`	`y/n`	Channel nutrient balance by stream order	(gated in `calsoft_control.f90`)
`res`	`y/n`	Reservoir budgets by reservoir	(gated in `calsoft_control.f90`)

If any switch is non-default the master flag `cal_soft` is set to `"y"` and `calsoft_control` is called instead of (or alongside) the normal simulation.

HRU water balance

`calsoft_hyd.f90` is the full water-balance loop. It iterates over land-use regions, reads the regional water-balance targets (`water_balance.sft`) and the per-parameter perturbation envelope (`wb_parms.sft`), and adjusts the active parameter set until each component is within tolerance. The components and their helper routines are:

- PET, `calsoft_hyd_bfr_pet.f90`
- ET, `calsoft_hyd_bfr_et.f90`
- Surface runoff, `calsoft_hyd_bfr_surq.f90`
- Lateral flow, `calsoft_hyd_bfr_latq.f90`
- Percolation, `calsoft_hyd_bfr_perc.f90`

The shared baseflow ratio loop is `calsoft_hyd_bfr.f90`.

Adjustable parameters for the upland water balance are read into `ls_prms` (`type soft_calib_parms`). Each entry carries a parameter name, the change type (`absval`, `abschg`, or `pctchg`), and `neg`/`pos` perturbation limits with absolute floor/ceiling values.

Channel sediment

`ch_sed_budget.sft` carries the per-stream-order sediment budget targets. `ch_sed_parms.sft` carries the perturbation envelope for channel sediment parameters, read into `ch_prms`. The channel sediment block is gated in `calsoft_control.f90` but the call to a `calsoft_chsed` driver is commented out in the current source; only the per-order accounting loop runs.

Plant growth

`plant_parms.sft` defines the plant parameter envelope (LAI potential, harvest index) and `plant_gro.sft` carries the per-region plant target table. The driver is `calsoft_plant.f90`; `calsoft_plant_zero.f90` zeros the running statistics between iterations. After soft calibration writes the new plant parameter values, `pl_write_parms_cal` dumps a fresh `plants.plt` and a `hydrology-cal.hyd` snapshot.

Outputs

`calsoft_control.f90` writes:

- `hydrology-cal.hyd` (unit 5001) with per-HRU hydrology parameters after the run.
- A channel sediment summary (unit 5000) when `chsed = y`.

These outputs are intended to be copied back into the model inputs to lock in the soft-calibrated values.

Input files

File	Read by	Purpose
`codes.sft`	`calsoft_read_codes.f90`	Component switches.
`wb_parms.sft`	(read in `calsoft_init`)	Upland water-balance parameter envelope.
`water_balance.sft`	(read in `calsoft_init`)	Regional water-balance targets (SR, LF, PC, ET, TF as fractions of precipitation).
`ch_sed_budget.sft`	(read in `calsoft_init`)	Channel sediment budget targets by stream order.
`ch_sed_parms.sft`	(read in `calsoft_init`)	Channel sediment parameter envelope.
`plant_parms.sft`	(read in `calsoft_init`)	Plant parameter envelope.
`plant_gro.sft`	(read in `calsoft_init`)	Per-region plant yield targets.

All `*.sft` slots are declared in `input_file_module.f90`, `type input_chg`. Set the slot to `null` (or remove the file) to skip a component.

Related pages

- Hard calibration. Use after soft calibration to fit remaining parameters to observed time series.
- Parameter change files. `cal_parms.cal` defines the parameter universe and bounds shared with hard calibration.

Hard calibration applies a list of named parameter changes from `calibration.cal`, re-initializes all model objects, then re-runs the time loop. Each change names a parameter, a change type (`absval`, `abschg`, `pctchg`), a value, a list of objects (HRUs, channels, aquifers, plants, …) to apply the change to, and optional conditions. The parameter universe and the absolute min/max for each parameter are defined in `cal_parms.cal`.

Source

- Driver: src/calhard_control.f90
- Parameter selector: src/cal_parm_select.f90
- Change application with conditions: src/cal_conditions.f90
- Parameter database reader: src/cal_parm_read.f90
- Change list reader: src/cal_parmchg_read.f90

Workflow

1. The model is run once normally. Output baseline is captured.
2. The user edits `calibration.cal` with the parameter changes to test (or an external driver writes it).
3. SWAT+ is re-launched. During startup, `cal_parm_read` loads `cal_parms.cal` into the `cal_parms` array and `cal_parmchg_read` loads `calibration.cal` into the `cal_upd` array. Each `cal_upd` entry is crosswalked to its `cal_parms` entry by name to pick up the absolute bounds and object type.
4. Inside the simulation, `cal_conditions` is called once. For every entry in `cal_upd`, it evaluates the conditions, then walks the element list and calls `cal_parm_select` for each affected object. `cal_parm_select` does the actual write into the live model state (HRU, channel, aquifer, plant, soil layer, climate series).
5. The master switch `cal_hard = "y"` triggers a second pass: `calhard_control` calls `re_initialize` then `time_control`, re-running the full simulation with the modified parameters.

The change type drives how the value is applied to the existing parameter:

`chg_typ`	Effect
`absval`	The new value replaces the existing one.
`abschg`	The value is added to the existing one (signed).
`pctchg`	The value is interpreted as a percent change.

After every change the result is clamped to the `[absmin, absmax]` range from `cal_parms.cal`.

Object types

`cal_parms.cal` tags each parameter with an object type (`ob_typ`). The type determines which collection in the model state is iterated when the change is applied:

Code	Collection	Examples
`hru`	HRUs	`cn2`, `ovn`, `lat_ttime`, `canmx`
`hlt`	`hru_lte`	parameters of the simplified HRU
`sol`	Soil layers within an HRU (`lyr1`/`lyr2` select the range)	`awc`, `bd`, `k`
`lyr`	Soil layers	layer-targeted variants
`plt`	Plants	`lai_pot`, `harv_idx`, `epco`, `phu_mat`
`aqu`	Aquifers	`alpha`, `revap_co`, `flo_min`
`cha`, `sdc`, `rte`	Channels (legacy, degraded, routed)	channel routing and sediment parameters
`swq`	Channel water quality	nutrient parameters
`res`	Reservoirs	`res_pvol`, surface area parameters
`rdt`	Reservoir decision tables	`drawdown_days`, `withdraw_rate`
`ru`	Routing units	RU-level parameters
`bsn`	Basin	basin-wide scalars in `parameters.bsn`
`pcp`, `tmp`, `cli`	Climate gauges and series	`precip`, `temp` adjustments over date ranges
`gwf`	gwflow cells	gwflow parameters

The list above is taken from the `select case (cal_parms(ipar)%ob_typ)` block in `cal_parmchg_read.f90` and the dispatch in `cal_conditions.f90`.

Re-initialization

`calhard_control.f90` calls `re_initialize` before re-running. This resets simulation state (water content, plant state, accumulators) so the second run is comparable to the baseline rather than continuing from baseline end-state.

Related pages

- Parameter change files. Column-level format of `cal_parms.cal` and `calibration.cal`.
- Conditions and regionalization. Limiting a change to HRUs that match a condition.
- Sensitivity workflow. Driving hard calibration from an external sampler.
- Soft calibration. Adjust process outputs toward regional targets before fitting to time series.

Purpose

`cal_parms.cal` is the dictionary of every parameter that calibration can touch: its name, the object type it lives on, and the absolute range it may be moved through. `calibration.cal` is the change list applied during a hard-calibration run: each row picks a parameter from the dictionary, sets a change type and magnitude, optionally restricts the change to a list of objects, and optionally attaches conditions. Both files feed the same parameter selector (`cal_parm_select`) used by hard calibration and by parameter changes inside soft calibration.

Source

- `cal_parms.cal` reader: `src/cal_parm_read.f90`
- `calibration.cal` reader: `src/cal_parmchg_read.f90`
- Type definitions: `src/calibration_data_module.f90`, `type calibration_parameters` (rows of `cal_parms.cal`) and `type update_parameters` (rows of `calibration.cal`).
- File slot names: `src/input_file_module.f90`, `type input_chg`. Defaults: `cal_parms = "cal_parms.cal"`, `cal_upd = "calibration.cal"`.

cal_parms.cal

`cal_parm_read.f90` reads:

- Line 1: title (skipped).
- Line 2: integer `mchg_par`, the number of parameter records that follow.
- Line 3: header (skipped).
- Lines 4 .. 3 + `mchg_par`: one record per parameter, read list-directed into `cal_parms(i)`.

Record layout (`type calibration_parameters`):

#	Field	Type	Description
1	`name`	char(25)	Parameter name. Must match a case in `cal_parm_select.f90`. Examples: `cn2`, `awc`, `alpha`, `epco`, `lat_ttime`.
2	`ob_typ`	char(25)	Object type the parameter is attached to: `hru`, `hlt`, `sol`, `lyr`, `plt`, `aqu`, `cha`, `sdc`, `rte`, `swq`, `res`, `rdt`, `ru`, `bsn`, `pcp`, `tmp`, `gwf`. Determines which collection is iterated.
3	`absmin`	real	Lower bound. Any change is clamped to this value.
4	`absmax`	real	Upper bound. Any change is clamped to this value.
5	`units`	char(25)	Units string for the parameter (`mm`, `m`, `days`, `null`, …). Documentation only; the reader does not interpret it.

Example

`refdata/Osu_1hru/cal_parms.cal`:

```
cal_parms.cal: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
186
name                        obj_typ       abs_min       abs_max             units
cn2                             hru      35.00000      95.00000              null
cn3_swf                         hru       0.00000       1.00000              null
usle_p                          hru       0.00000       1.00000              null
ovn                             hru       0.01000      30.00000              null
elev                            hru       0.00000    5000.00000                 m
slope                           hru       0.00010       0.90000               m/m
slope_len                       hru      10.00000     150.00000                 m
lat_ttime                       hru       0.50000     180.00000              days
```

The count line on line 2 (`186`) drives the allocation of `cal_parms(0:mchg_par)`. The header on line 3 (`name`, `obj_typ`, `abs_min`, `abs_max`, `units`) is skipped; values are read by position.

calibration.cal

`cal_parmchg_read.f90` reads:

- Line 1: title (skipped).
- Line 2: integer `mcal`, the number of change records.
- Line 3: header (skipped).
- Then `mcal` change records, each followed by `conds` condition lines (often zero).

Change record

Each change record is one logical line, read list-directed in two passes. The first pass reads the fixed prefix; if the prefix indicates an object list is present, the line is backspaced and re-read to pick up the integers.

Fixed prefix:

#	Field	Type	Description
1	`name`	char(25)	Parameter name. Must match a `name` in `cal_parms.cal`. The reader crosswalks this to `num_db`.
2	`chg_typ`	char(16)	Change operator. One of `absval` (replace), `abschg` (add), `pctchg` (percent change).
3	`val`	real	Magnitude of the change, interpreted per `chg_typ`.
4	`conds`	int	Number of condition lines that follow this record. May be zero.
5	`lyr1`	int	First soil layer in range (0 means all layers). For reservoir decision tables, reused as a storage-zone flag.
6	`lyr2`	int	Last soil layer in range (0 means through last layer).
7	`year1`	int	First year (for climate parameter changes).
8	`year2`	int	Last year (for climate parameter changes).
9	`day1`	int	First day of year (for climate parameter changes).
10	`day2`	int	Last day of year (for climate parameter changes).
11	`nspu`	int	Number of object indices listed after this prefix. `0` means apply to every object of this parameter's `ob_typ`.

If `nspu > 0`, the same line continues with `nspu` integers giving the object indices to update. When `nspu == 0` the reader allocates a `num` array that lists every object of the matching type (every HRU, every channel, every plant, and so on) so the change applies globally.

Condition lines

A condition line takes one of two forms, distinguished by the literal token in column 1:

- `range <var> <val1> <val2>`. Restricts the change to objects whose `<var>` falls between `val1` and `val2`. Used in `cal_conditions.f90` for `res_pvol`.

- `<var> <alt> <targ> <targc>`. Read as a `calibration_conditions` record. Used for categorical filters: `hsg`, `texture`, `plant`, `pl_class`, `landuse`, `cal_group`, `res_typ`.

See Conditions and regionalization for the supported variable list.

Example

`refdata/Osu_1hru/calibration.cal`:

```
  hru-new.cal developed from soft data calibration
    13
 NAME           CHG_TYP                  VAL    CONDS  LYR1  LYR2   YEAR1  YEAR2   DAY1   DAY2  OBJ_TOT
cn2          pctchg                 7.714000E+00  0          0          0          0          0
petco        pctchg                -5.273950E+01  0          0          0          0          0
latq_co      pctchg                -3.732030E+01  0          0          0          0          0
alpha        pctchg                 6.859000E+00  0          0          0          0          0
epco         pctchg                 6.534100E+01  0          0          0          0          0
awc          pctchg                -6.256550E+01  0          0          0          0          0
esco         pctchg                 3.305500E+01  0          0          0          0          0
perco        pctchg                 1.434000E+00  0          0          0          0          0
cn3_swf      pctchg                -7.710000E-02  0          0          0          0          0
canmx        pctchg                -6.680230E+01  0          0          0          0          0
lat_len      pctchg                -6.241200E+00  0          0          0          0          0
revap_co     pctchg                -5.854530E+01  0          0          0          0          0
surlag       pctchg                -2.125720E+01  0          0          0          0          0
```

In this file every record has `conds = 0` and `nspu = 0`, so each change applies to every HRU. The trailing integer count is `11` per row: `conds`, `lyr1`, `lyr2`, `year1`, `year2`, `day1`, `day2`, and `nspu` (eight integers), padded by the editor with extra zeros that the list-directed reader ignores.

Carbon parameters

SWAT+ wires 40 carbon-related parameters in `cal_parm_select.f90`. Each entry below maps a calibration name to its target field, suggests a working min/max range, and notes the object type (`bsn`, `lyr`, `plt`, `hru`) that must appear in the `ob_typ` column of `cal_parms.cal`. Bounds come from the sptAPI bundled `cal_parms.cal` (261 entries), with the temperature triple tightened to physical bounds to physically realistic ranges.

Basin scalars from `carbon.bsn` (18 entries)

name	ob_typ	units	abs_min	abs_max	description
init_seq	bsn	fraction	0.5	1.0	initial sequestration fraction (org_frac%frac_seq)
init_microb	bsn	fraction	0.0	0.1	initial microbial-humus fraction (org_frac%frac_hum_microb)
init_slow	bsn	fraction	0.2	0.6	initial slow-humus fraction (org_frac%frac_hum_slow)
init_passive	bsn	fraction	0.2	0.6	initial passive-humus fraction (org_frac%frac_hum_passive)
koc_c	bsn	none	500	1500	KOC for C transport (cb_wtr_coef%prmt_21)
solc_ratio	bsn	0..1	0.1	1.0	runoff:percolate soluble C ratio (cb_wtr_coef%prmt_44)
manure_c_frac	bsn	none	0.0	1.0	manure C partition between fresh and stable pools (man_coef%rtof)
bio_consol	bsn	none	0.05	0.5	biological consolidation factor (bio_consf)
till_consol	bsn	none	0.05	0.5	tillage consolidation factor (till_consf)
sfc_rsd_photodeg	bsn	1/day	0.0	0.05	surface residue photodegradation rate (photo_degrade_factor)
n_act_frac	bsn	fraction	0.005	0.04	fraction of organic N in active humus pool (n_act_frac)
cnr_cap	bsn	none	100	1000	upper cap on residue C:N ratio
cnr_ref	bsn	none	10	50	reference C:N ratio where decomp factor = 1
cpr_cap	bsn	none	1000	10000	upper cap on residue C:P ratio
cpr_ref	bsn	none	100	400	reference C:P ratio where decomp factor = 1
t_cbn_min	bsn	deg C	-5	5	minimum temperature bound for tmpf2 (org_con%tn); tightened to physical bounds
t_cbn_opt	bsn	deg C	25	40	optimum temperature for tmpf2 (org_con%top); tightened to physical bounds
t_cbn_max	bsn	deg C	45	55	maximum temperature bound for tmpf2 (org_con%tx); tightened to physical bounds

Note

The `min < opt < max` ordering invariant is not enforced by `cal_parms.cal` itself; calibration tools may still produce invalid combinations. The `fcgd` clamp ensures invalid orderings degrade gracefully (return 0) rather than crashing.

Per-layer fields from `carbon_lyr.bsn` (11 entries)

Each entry addresses `carbdb(ly)` or `org_allo(ly)` using the `lyr1`/`lyr2` slot of the `calibration.cal` record. Bounds are guarded to `[1, size(carbdb)]` inside the `case` branch, so out-of-range layer indices are silently no-op.

name	ob_typ	units	abs_min	abs_max	description
hp_rate	lyr	1/day	1.0e-06	1.0e-04	passive humus turnover rate (carbdb(ly)%hp_rate)
hs_rate	lyr	1/day	1.0e-05	1.0e-03	slow humus turnover rate (carbdb(ly)%hs_rate)
microb_rate	lyr	1/day	0.005	0.05	microbial biomass turnover rate (carbdb(ly)%microb_rate)
meta_rate	lyr	1/day	0.01	0.1	metabolic litter turnover rate (carbdb(ly)%meta_rate)
str_rate	lyr	1/day	0.005	0.05	structural litter turnover rate (carbdb(ly)%str_rate)
microb_top_rate	lyr	1/day	0.005	0.05	top-layer microbial activity adjustment (carbdb(ly)%microb_top_rate)
hs_hp	lyr	none	0.01	0.2	slow-to-passive humus allocation (carbdb(ly)%hs_hp)
a1co2	lyr	fraction	0.3	0.8	metabolic litter to CO2 (org_allo(ly)%a1co2)
asco2	lyr	fraction	0.3	0.8	slow humus to CO2 (org_allo(ly)%asco2)
apco2	lyr	fraction	0.3	0.8	passive humus to CO2 (org_allo(ly)%apco2)
abco2	lyr	fraction	0.0	0.8	microbial biomass to CO2 (org_allo(ly)%abco2)

Biomixing and tillage curve coefficients (6 entries)

Curve fit coefficients used by `mgt_tillfactor.f90` tillf option 4. The `zz_` prefix that the older code carried was dropped to match the column names in `carbon.bsn`.

name	ob_typ	units	abs_min	abs_max	description
bmix_a	bsn	none	1.0	5.0	biological mixing coefficient A
bmix_b	bsn	none	1.0	10.0	biological mixing coefficient B
bmix_c	bsn	none	-10.0	-1.0	biological mixing coefficient C
tillmix_a	bsn	none	1.0	5.0	tillage mixing coefficient A
tillmix_b	bsn	none	5.0	25.0	tillage mixing coefficient B
tillmix_c	bsn	none	-10.0	-1.0	tillage mixing coefficient C

Plant carbon parameters (5 entries)

Carbon-relevant parameters on the plant database (`pldb(ielem)`). These were not previously exposed even though plant biomass drives the whole NPP and residue chain.

name	ob_typ	units	abs_min	abs_max	description
bio_e	plt	(kg/ha)/(MJ/m^2)	10.0	60.0	biomass-energy ratio (drives NPP)
hvsti	plt	fraction	0.1	0.8	harvest index (controls residue vs grain C split)
rdmx	plt	m	0.5	3.0	maximum root depth
t_opt	plt	deg C	15	35	optimum plant growth temperature
t_base	plt	deg C	0	15	base plant growth temperature

The existing `usle_c` plant parameter is retained from earlier releases.

> **Note**
>
> For the `init_*` parameters to take effect, `proc_cal` must run **before** `soil_nutcarb_init`. The current `main.f90` already orders these correctly; do not reorder. Previously the order was reversed and `init_*` calibration was a silent no-op because the pools were already sized using pre-cal values.

Related files

- `carbon.bsn`. Basin-wide carbon scalar inputs whose fields these parameters target.
- `carbon_lyr.bsn`. Per-layer carbon inputs.
- Hard calibration. How these files are consumed during a calibration run.
- Conditions and regionalization. Condition-line vocabulary.

A change in `calibration.cal` can carry condition lines that restrict where the change is applied. The conditions are tested object by object in `cal_conditions.f90`. If every condition evaluates true for a given object, the change is passed to `cal_parm_select` for that object; otherwise the object is skipped.

Source

- Condition evaluation: `src/cal_conditions.f90`
- Condition reader: `src/cal_cond_read.f90`
- Parameter dispatch: `src/cal_parm_select.f90`
- Condition record type: `src/calibration_data_module.f90`, `type calibration_conditions` and `type update_conditional`.

Two condition syntaxes

`cal_parmchg_read.f90` peeks at the first token of each condition line:

- If the token is the literal word `range`, the line is read as `range <var> <val1> <val2>`. The condition holds when the value of `<var>` for the current object falls inside `[val1, val2]`. `val1` and `val2` are stored on the parent change record (shared across that record's range conditions).
- Otherwise the line is read as a `calibration_conditions` record: `<var> <alt> <targ> <targc>`. `targ` is a real, `targc` is a string. The reader stores all four fields but `cal_conditions.f90` only uses `var`, `targc`, and `targ` depending on the variable.

Supported variables

The `select case (cal_upd(ichg_par)%cond(ic)%var)` block in `cal_conditions.f90` accepts:

Variable	Object type	Form	Semantics
`res_pvol`	reservoirs	range	Reservoir principal storage volume (compared in units of 10000).
`hsg`	HRU soils	categorical (`targc`)	Hydrologic soil group must equal `targc` (`A`, `B`, `C`, `D`).
`res_typ`	reservoirs	categorical (`targc`)	Reservoir decision-table name must equal `targc`.
`texture`	HRU soils	categorical (`targc`)	Soil texture string must equal `targc`.
`plant`	HRU plant community	categorical (`targc`)	The HRU's plant community must contain a plant named `targc`.
`pl_class`	HRU plants	categorical (`targc`)	Plant class for the HRU must equal `targc`.
`landuse`	HRU land use management	categorical (`targc`)	`hru%land_use_mgt_c` must equal `targc`.
`cal_group`	HRU calibration tag	categorical (`targc`)	`hru%cal_group` must equal `targc`.

A change with multiple conditions implements an AND: all must hold for the object to be modified.

Regionalization

Two patterns produce a regional effect:

- **Implicit through `cal_group`.** Tag every HRU with a `cal_group` name (set on the HRU input). Then a change with a `cal_group <name>` condition runs only on that group. This is how named region-based hard calibration is typically done.
- **Explicit object list.** Put the HRU (or channel, or aquifer) indices directly on the change record (`nspu > 0`). No conditions needed. This is what soft calibration writes when it commits its perturbations.

The two can combine: an explicit object list with one or more conditions further filters the list.

Decision-table conditional updates

`cal_cond_read.f90` reads a separate file, `scen_dtl.upd`, that schedules updates by decision table. Each row gives a maximum number of times the table may fire, a table type, and the name of a `dtbl` entry in `conditional.ctl`. The reader crosswalks the table name into `dtbl_scen` and stores the result in `upd_cond` (`type update_conditional`). The same decision-table machinery used elsewhere in SWAT+ (land-use change scenarios, management triggers) is reused here.

Parameter dispatch

Once conditions pass, the change is sent to `cal_parm_select.f90` with the parameter name, change type, magnitude, absolute bounds, the target element, and the soil layer (for layer-targeted soil parameters). `cal_parm_select` is a long `select case (chg_parm)` that writes the change into the live model state via `chg_par` (the change operator) and calls the relevant re-initializer (`curno` for `cn2`, `soil_awc_init` for `awc`, and so on).

Related pages

- Parameter change files. The columns that hold these conditions on disk.
- Hard calibration. When conditions are evaluated in the calibration run.

SWAT+ has no built-in sensitivity analysis. Sensitivity is run by an external tool that writes `calibration.cal`, launches the SWAT+ executable, reads an objective value from the simulation output, and repeats. The parameter list and bounds are taken from the same `cal_parms.cal` used by hard calibration.

Pattern

1. Pick the parameters to test. Pull their absolute ranges from `cal_parms.cal` or override them with a tighter calibration range. Each parameter is one dimension of the sample space.
2. Generate a sample matrix using a sensitivity method (Sobol, FAST, Morris, LH-OAT, …). Each row of the matrix is one parameter vector.
3. For each row: write `calibration.cal` with one `pctchg` (or `absval`, `abschg`) record per parameter, run SWAT+, read the chosen objective (NSE, KGE, percent bias, total flow, …) from the output files, and record it.
4. Feed the sample matrix and the model performance vector into the sensitivity method to compute indices: first-order, total-order, second-order interaction, or elementary effects depending on the method.

The expensive step is the simulation loop. A sample size in the hundreds to low thousands is common.

sensitivityAPI

`sensitivityAPI` is a command-line tool that implements steps 2 and 4 of the pattern. It operates on a SWAT+

TxtInOut directory: it reads a parameter CSV (`par_data.stb`), writes a sample matrix (`par_sample.stb`), and after the user has run the simulations and collected performance values into `perf_report.stb`, computes sensitivity indices.

Supported methods:

Method	Sampling	Indices
`FAST`	frequency-based	S1, ST
`sobol`	Saltelli quasi-random	S1, ST, S2
`RBD_FAST`	Latin Hypercube	S1
`DMIM`	Latin Hypercube	S1 (delta)
`lhoat`	LH-OAT hybrid	S1 (elementary effects)

Usage:

```
sensitivityAPI generate_sample sobol /path/to/txtinout 1024
# run the simulations here, fill perf_report.stb
sensitivityAPI analyse_sensitivity sobol /path/to/txtinout
```

The orchestration (writing `calibration.cal` per row, launching SWAT+, parsing the output) is done by a driver such as sptAPI or spToolboX. `sensitivityAPI` is only the sampler and the post-processor.

Roll your own

Any language with a numerical library can do the same. The minimum loop:

1. Parse `cal_parms.cal` to get bounds for the parameters you care about.
2. Generate `N` parameter vectors with a Python (`SALib`), R (`sensitivity`), or in-house sampler.
3. For each vector, overwrite `calibration.cal` with the corresponding `pctchg` records and call the SWAT+ binary with the TxtInOut directory as the working directory.
4. Read the metric from `channel_sd_day.txt` (or whichever output your objective uses), record it.
5. Hand the matrix and the vector to your sensitivity routine.

Related pages

- Parameter change files. Format of `cal_parms.cal` and `calibration.cal`.
- Hard calibration. The mechanism each sensitivity run drives, one parameter vector at a time.